

## Análisis de programa RentaMovil

### 1. Requisitos funcionales

| Información                  | Datos necesarios                                                  |
|------------------------------|-------------------------------------------------------------------|
| Datos generales del vehículo | Placa, marca, modelo y tarifa diaria base.                        |
| Estado del vehículo          | Si está rentado y cuántos días tiene asignados a su renta actual. |
| Datos del automóvil          | Capacidad de personas y si tiene transmisión automática.          |
| Datos de la motocicleta      | Cilindraje en centímetros cúbicos.                                |
| Datos del camión             | Capacidad de carga en toneladas.                                  |
| Inventario                   | Colección de vehículos registrados.                               |
| Ingresos                     | Total acumulado de las rentas confirmadas durante la ejecución.   |

## 2. Operaciones necesarias

| Operación          | Comportamiento esperado                                                              | Condiciones para realizarla                                                                                                       |
|--------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Registrar vehículo | Incorporar un automóvil, motocicleta o camión al inventario.                         | Placa no vacía y única; marca y modelo no vacíos; tarifa positiva; características específicas positivas; categoría seleccionada. |
| Buscar vehículo    | Recuperar un vehículo mediante su placa.                                             | Placa no vacía. Si no existe, informar que no se encontró.                                                                        |
| Consultar vehículo | Mostrar categoría, marca, modelo, tarifa base y disponibilidad.                      | El vehículo debe existir.                                                                                                         |
| Cotizar alquiler   | Calcular el importe correspondiente a los días solicitados.                          | Vehículo existente y días enteros mayores que cero. No modifica su estado.                                                        |
| Confirmar alquiler | Registrar los días, marcar el vehículo como rentado y sumar el importe al acumulado. | Vehículo existente, disponible, días válidos y confirmación del usuario.                                                          |
| Retornar vehículo  | Marcarlo como disponible y establecer sus días de renta en cero.                     | El vehículo debe existir y estar rentado.                                                                                         |
| Consultar resumen  | Mostrar cantidad registrada, disponibles, rentados e ingresos acumulados.            | Debe consultar el estado actualizado del inventario.                                                                              |
| Navegar y salir    | Abrir las distintas pantallas o finalizar la aplicación.                             | No requiere condiciones de negocio adicionales.                                                                                   |

### 3. Clases y sus responsabilidades

| Grupo    | Clases                                                | Responsabilidad                                                                      |
|----------|-------------------------------------------------------|--------------------------------------------------------------------------------------|
| Modelo   | Vehiculo, Automovil, Moto, Camion                     | Representar los vehículos, proteger su estado y calcular sus tarifas.                |
| Gestión  | GestorDatos                                           | Administrar el inventario, coordinar alquileres y devoluciones, y acumular ingresos. |
| Interfaz | Inicio, Registro, Consultas, Renta, Retornar, Resumen | Recibir entradas, solicitar operaciones al gestor y presentar resultados.            |

#### Clase Vehículo

##### Atributos

| Visibilidad | Atributo     | Tipo    | Propósito                                                       |
|-------------|--------------|---------|-----------------------------------------------------------------|
| —           | placa        | String  | Identificar el vehículo. No se modifica después de registrarlo. |
| —           | marca        | String  | Conservar la marca.                                             |
| —           | modelo       | String  | Conservar el modelo.                                            |
| —           | tarifaDiaria | double  | Guardar el precio diario base.                                  |
| —           | rentado      | boolean | Indicar si tiene un alquiler activo.                            |
| —           | diasRenta    | int     | Guardar los días del alquiler actual; cero si está disponible.  |

## Constructor y métodos

| Visibilidad | Firma                                                                    | Propósito y resultado                                                                                               |
|-------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| #           | Vehiculo(String placa, String marca, String modelo, double tarifaDiaria) | Inicializar datos generales válidos. Establecer rentado = false y diasRenta = 0.                                    |
| +           | getPlaca(): String                                                       | Devolver la placa.                                                                                                  |
| +           | getMarca(): String                                                       | Devolver la marca.                                                                                                  |
| +           | getModelo(): String                                                      | Devolver el modelo.                                                                                                 |
| +           | getTarifaBasica(): double                                                | Devolver la tarifa diaria sin recargos.                                                                             |
| +           | getRentado(): boolean                                                    | Indicar si está rentado.                                                                                            |
| +           | getDias(): int                                                           | Devolver los días registrados.                                                                                      |
| +           | getTipo(): String — abstracto                                            | Devolver el nombre de la categoría.                                                                                 |
| +           | calcularTarifaTentativa(int dias): double — abstracto                    | Validar días positivos y calcular una cotización según la categoría, sin modificar el vehículo.                     |
| +           | calcularTarifa(): double                                                 | Devolver el costo de la renta actual mediante calcularTarifaTentativa(diasRenta); devolver cero si está disponible. |
| +           | iniciarRenta(int dias): void                                             | Validar disponibilidad y días positivos; registrar los días y marcarlo como rentado.                                |
| +           | retornar(): void                                                         | Validar que esté rentado; marcarlo como disponible y reiniciar sus días.                                            |

### Subclases

| Clase     | Atributo privado    | Tipo    | Propósito                                                                     |
|-----------|---------------------|---------|-------------------------------------------------------------------------------|
| Automovil | capacidadPersonas   | int     | Número máximo de personas; debe ser positivo.                                 |
| Automovil | automatico          | boolean | Determinar si corresponde el recargo diario de 50.                            |
| Moto      | cilindraje          | double  | Determinar si supera los 250 cc; debe ser positivo.                           |
| Camion    | capacidadTransporte | double  | Capacidad en toneladas utilizada para calcular el recargo; debe ser positiva. |

### Clase GestorDatos – Atributos

| Visibilidad | Atributo       | Tipo                        | Propósito                                     |
|-------------|----------------|-----------------------------|-----------------------------------------------|
| –           | INSTANCIA      | static final<br>GestorDatos | Mantener una única instancia compartida.      |
| –           | vehiculos      | final List<Vehiculo>        | Conservar el inventario.                      |
| –           | totalAcumulado | double                      | Sumar los importes de alquileres confirmados. |

## Clase GestorDatos – Metodos

| Visibilidad | Firma                                             | Propósito y resultado                                                                                      |
|-------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| –           | GestorDatos()                                     | Crear la lista, iniciar el total en cero y cargar los vehículos iniciales una sola vez.                    |
| –           | cargarDatosIniciales(): void                      | Construir y registrar los vehículos de ejemplo.                                                            |
| +           | getInstancia(): GestorDatos —<br>estático         | Devolver la instancia compartida.                                                                          |
| +           | getVehiculos(): List<Vehiculo>                    | Devolver una copia no modificable de la lista para consultar el inventario.                                |
| +           | registrarVehiculo(Vehiculo<br>vehiculo): void     | Validar que no sea nulo y que su placa no esté registrada; agregarlo.                                      |
| +           | buscarPorPlaca(String placa):<br>Vehiculo         | Buscar sin distinguir mayúsculas y minúsculas, ignorando espacios exteriores; devolver el vehículo o null. |
| +           | cotizar(String placa, int dias):<br>double        | Validar existencia y días; solicitar al vehículo su cálculo.                                               |
| +           | confirmarRenta(String placa, int<br>dias): double | Validar, recalcular el importe, iniciar la renta y acumular el ingreso. Devolver el monto confirmado.      |
| +           | retornarVehiculo(String placa):<br>void           | Buscar el vehículo y solicitar su devolución.                                                              |
| +           | contarDisponibles(): int                          | Devolver la cantidad de vehículos disponibles.                                                             |
| +           | contarRentados(): int                             | Devolver la cantidad de vehículos rentados.                                                                |
| +           | getTotalAcumulado(): double                       | Devolver el ingreso acumulado actualizado.                                                                 |

#### 4. Elementos comunes y variables

La herencia evita repetir placa, marca, modelo, tarifa, disponibilidad y días en cada categoría.

Los atributos específicos permanecen en sus subclases porque solo tienen sentido para ellas. Por ejemplo, el cilindraje no pertenece a un camión ni la capacidad en toneladas a un automóvil.

Las decisiones de cobro se ubican en `calcularTarifaTentativa(int dias)` de cada subclase. `GestorDatos` utiliza una referencia de tipo `Vehiculo` y llama al método sin preguntar si se trata de una moto, un automóvil o un camión. Java ejecuta la implementación correspondiente mediante polimorfismo.

El método `getTipo()` también evita repetir cadenas de `instanceof` en los formularios.

Colaboración para completar un alquiler

1. El usuario escribe la placa y los días en Renta.
2. El formulario valida que los días puedan convertirse a un entero.
3. Para cotizar, solicita `gestor.cotizar(placa, dias)`.
4. El gestor localiza el vehículo y le pide calcular el importe.
5. El formulario presenta la cotización y solicita confirmación.
6. Si el usuario acepta, llama a `gestor.confirmarRenta(placa, dias)`.
7. El gestor verifica nuevamente disponibilidad y días, y recalcula el importe con esos mismos datos.
8. El vehículo registra su alquiler mediante `iniciarRenta(dias)`.
9. El gestor suma el importe y lo devuelve a la interfaz.

La confirmación no utiliza un costo almacenado por una cotización anterior. Esto evita cobrar un importe correspondiente a otra placa o a otra cantidad de días.

## 5. Incorporación de una nueva categoría

Si la empresa incorpora, por ejemplo, un autobús, será necesario:

- Crear `Autobus` extends `Vehiculo`.
- Definir sus atributos particulares y su constructor.
- Implementar `getTipo()` y `calcularTarifaTentativa(int dias)`.
- Añadir la categoría y sus campos al formulario Registro.
- Actualizar la carga inicial si se requieren ejemplos de autobuses.

Las operaciones de búsqueda, alquiler, devolución y resumen seguirán trabajando con `Vehiculo`, por lo que no necesitarán nuevas condiciones por categoría.